

홍철민

Java/Kotlin·Spring 백엔드 개발자 백엔드 경력 3년 7개월

연성대학교 컴퓨터소프트웨어 전문학사 졸업

Email zxcbnm6404@gmail.com | GitHub github.com/Hong1008 | Portfolio hong1008.github.io/portfolio/



프로필 Profile

결제, 비동기 메시징과 AWS 배포를 구현해 온 백엔드 개발자입니다.

운영 서비스에서 중복 요청과 재실행, 외부 연동 이후의 부분 실패를 DB 제약과 명시적인 상태 흐름으로 다뤘습니다.

신규 시스템의 도메인·데이터 책임을 나누고, Spring 애플리케이션을 위한 AWS 실행·배포 환경을 함께 구현했습니다.

핵심 기술 Skills

Backend	Java · Kotlin · Spring Boot · Spring MVC · JPA · QueryDSL
Data & Messaging	MySQL · Redis · AWS SQS · Spring Batch
Infrastructure	AWS · Docker · Jenkins · ECS Fargate
Recent Expansion	Python · FastAPI · RAG · MCP

경력 Professional Experience

샤플앤크퍼니 · HADA 개발팀

2022.11-2023.08

백엔드 개발자 · 운영

팀 백엔드 2명, 리드 개발자 1명, 프론트엔드 1명, PM·기획 2명으로 구성된 6명 팀

현장 관리 서비스의 메일 발송, 점검 보고서와 현장 이슈 관리 백엔드를 담당했습니다.

기여 세 업무의 백엔드 구현을 직접 담당하고, 업무별 기획자·프론트엔드 개발자와 협업했습니다.

배치 재실행과 메시지 중복 전달을 분리한 메일 발송 구조

문제·제약	기존 네이버웍스 API는 서버의 일괄 발송에 적합하지 않았고, 배치 재실행과 SQS 메시지 재전달을 서로 다른 실패 조건으로 다뤄야 했습니다.
판단·구현	AWS SES로 전환하고 대상자를 선정하는 배치 서버와 전송을 담당하는 메일 서버를 SQS로 분리했습니다. 배치 단계에는 발송 기준일, 소비 단계에는 메일 트랜잭션 키를 포함한 DB UNIQUE 제약을 적용했습니다.
검증·결과	메일 발송 기능은 운영됐으며, 온보딩 상태와 발송 조건을 분리한 구조를 인보이스 메일에도 재사용했습니다.
책임·한계	두 단계의 DB 제약은 중복 처리를 방어하는 범위이며, 메일이 항상 한 번만 전달된다는 보장은 아닙니다.

보고서 생성 엔진 전환과 Kotlin DSL 공통화

문제·제약	POI 기반 점검 보고서 생성에서 지연이 발생했고 Workbook·Sheet·Row·Cell 처리 코드가 보고서마다 반복됐습니다.
판단·구현	생성 엔진을 FastExcel로 전환하고 반복 흐름을 Kotlin DSL 공통 빌더로 분리해 보고서 로직이 엑셀 라이브러리에 직접 의존하지 않도록 했습니다.
검증·결과	공통 빌더는 운영된 점검 보고서에서 사용됐고, 보고서별 구현은 같은 생성 구조를 재사용했습니다.
책임·한계	운영 환경의 전후 측정 근거를 확인할 수 없어 생성 시간 개선 수치는 사용하지 않습니다.
추가 기여	현장 이슈의 상태·변경 이력과 AWS S3 기반 다중 이미지 첨부·수정 API를 구현했습니다.

기술 Kotlin · Spring Boot · Spring Batch · MySQL · AWS SQS·SES · FastExcel · AWS S3

휴니크 · 개발팀

2021.09-2022.10

개발 리드·백엔드·인프라 · 출시 전 종료

팀 백엔드 2명과 프론트엔드 2명으로 구성된 4명 개발팀

기존 PHP 서비스와 공존할 매장 예약 서비스 젤로텍 v2를 별도의 Spring 시스템으로 개발했습니다.

기여 개발 리드로 신규 데이터 모델, 어드민·키오스크 API와 AWS 실행·배포 환경을 담당했습니다.

레거시 payment 구조를 복제하지 않은 신규 데이터 모델

문제·제약	기존 payment 테이블의 30개 이상 컬럼에 상품·이용권·프로모션·회원·수업·주문 정보가 결합돼 있었습니다.
판단·구현	기존 구조를 직접 변경하거나 복제하지 않고 신규 시스템 안에서 도메인별 데이터 책임을 나눴습니다. 기존 회원 인증 결과는 연동하되 Spring 시스템의 데이터와 API는 별도로 구성했습니다.
검증·결과	어드민·키오스크 API, JWT·Spring Security 인증과 메뉴별 권한을 구현하고 인증·예외·공통 응답을 멀티모듈 공통 영역으로 분리했습니다.
책임·한계	프로젝트는 출시 전에 종료돼 실제 데이터 마이그레이션, 운영 트래픽 전환과 사용자 성과는 검증하지 못했습니다.

신규 Spring 시스템의 AWS 실행·배포 환경 구성

문제·제약	신규 시스템을 기존 서비스와 독립적으로 실행하고 반복 배포할 AWS 환경과 배포 흐름이 필요했습니다.
판단·구현	ECS Fargate Task를 Private Subnet에 배치하고 ALB·RDS·ECR과 외부 통신 경로를 구성했습니다. Jenkins에서 Docker 이미지를 빌드해 ECR에 올리고 ECS로 배포하도록 연결했습니다.
검증·결과	ECS의 롤링 배포와 롤백 기능을 사용할 수 있게 했고, 프론트엔드 결과물은 S3와 CloudFront로 제공했습니다.
책임·한계	출시 전 종료된 환경이므로 실제 운영 트래픽의 배포 연속성이나 장애 대응 수준을 검증한 사례가 아닙니다.

기술 Java · Spring Boot · Spring Security · MySQL · Docker · Jenkins · AWS ECS·RDS·ALB

팀 통합 결제 시스템은 백엔드 개발자 본인 1명과 DBA 1명이 담당

PG와 Google Play·Apple 인앱 결제를 하나의 서비스 흐름으로 연결했습니다.

기여 애플리케이션 결제 상태 흐름과 복구 배치를 구현하고, Stored Procedure와 DB 처리 구조를 설계한 DBA와 협업했습니다.

외부 결제 승인과 내부 재화 제공을 분리한 상태 흐름

문제·계약 외부 결제 승인과 내부 재화 제공은 하나의 트랜잭션으로 묶을 수 없어 승인 이후의 부분 실패와 반복 요청을 구분할 필요가 있었습니다.

판단·구현 주문을 REQUESTED·PURCHASED·COMPLETED로 나누고, 애플리케이션의 주문 상태 검증과 결제 트랜잭션 식별키 UNIQUE 계약을 서로 다른 중복 방어선으로 사용했습니다.

검증·결과 Redis의 PURCHASED 정보를 약 300초 TTL로 관리하고 남은 TTL이 약 130초 이하인 미완료 주문을 다시 확인해 처리했습니다. ShedLock은 여러 WAS의 복구 스케줄러 동시 실행만 제한했습니다.

책임·한계 승인 후 DB 상태 저장 자체가 실패한 주문은 이 구조로 복구할 수 없으며, Stored Procedure와 DB 처리 구조는 DBA의 책임이었습니다.

추가 기여 Google Play 구독 연장·구매 확인과 Apple 영수증 검증을 구현하고, PG사별 결제 로직을 공통 인터페이스와 구현체로 분리했습니다.

기술 Java · Spring Boot · MySQL · Redis · ShedLock · Stored Procedure

지투이 · 개발팀

2020.02-2021.02

애플리케이션·백엔드 개발 · 사내·고객 시스템

응급의료 현장에서 사용하는 Android 키오스크 앱과 백엔드 API를 개발했습니다.

기여 Flutter 앱과 Spring Boot CRUD API를 함께 개발했습니다.

- 화면별로 반복되는 전역 오류 처리, 인증, 세션과 감사 로그를 공통 모듈로 구성했습니다.

기술 Java · Spring Boot · MySQL · Flutter

프로젝트 Selected Projects

KExcel · Kotlin DSL 기반 엑셀 생성 라이브러리

2026.05 · 개인 오픈소스

- 실무의 엑셀 생성 문제를 일반화해 Kotlin DSL과 ExcelDriver 추상화, 스트리밍·동시 쓰기 오용 감지를 구현했습니다.
- OpenJDK 21·512MB heap에서 100만 행을 3.396초에 생성하고 DSL 실행당 추가 할당량을 약 200KB에서 364B로 줄였으며, 동일 시트 동시 쓰기는 감지해 중단합니다.

<https://github.com/Hong1008/kexcel>

- 5인 팀의 PM·시스템 설계·MCP 구현을 맡아 결정론적 검색·상태 분류와 선택적 LLM 계층을 분리했습니다.
- 약 0.0328 범위의 RRF 점수가 0.5 임계값과 비교되던 오류를 수정하고 판정 후보·점수를 실행 기록으로 보존했으며, 결과는 법률 판단이 아닌 검토 후보입니다.

<https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN30-4th-2Team>

교육 · 자격 Training & Certification

교육	SK Networks Family AI 캠프 30기 · 실습 중심 교육 과정 수강	2026.04-현재
자격	SQL 개발자(SQLD) · 한국데이터산업진흥원	2026.06
자격	PCCE Python3 Lv4 · (주)그렙	2026.04